ЛР2 LNX-4 VTKM (Kernel Module)

Вопросы и ответы к защите

Вариант: **syscall, Linux header** — системный вызов, структура `task_struct` по PID

Данилов Тимофей Николаевич, P3331
Практик: Максимов Андрей Николаевич

09.11.2026

Откуда вопросы. Раздел 1 — официальный список «Вопросы на понимание» из вкладки «LNX-4: VTKM (Kernel Module)» документа «Задания ОС 2026». Раздел uВопросов (на самостоятельность) там пока пуст. Раздел 2 — вопросы, которые вероятны именно для моего варианта: их составил я сам, в официальном списке их нет. Раздел 3 — что спросят про мой код. Раздел 4 — команды для демонстрации, раздел 5 — материалы.

Формат защиты (из документа курса): практик случайно выбирает N вопросов на понимание и M uВопросов, возможно в другой формулировке. Если по ответам видно, что задание сделано не самостоятельно, сдача задания блокируется.

> **ПРОВЕРИТЬ НА СВОЁМ ЯДРЕ**
> На `home` стоит ядро 7.0. Часть ответов зависит от версии (режимы вытеснения, имена функций). Перед защитой сверь с исходниками своей версии на elixir.bootlin.com.

Содержание

1 Официальные вопросы на понимание 3
1.1 Драйверы устройств: отличие от обычных программ, ограничения и возможности, безопасность, сборка 3
1.2 Модули ядра Linux: создание и загрузка, отличие от программ в user space 3
1.3 Архитектура ОС: сложности разработки в ядре, нюансы работы с FPU 4
1.4 Режимы работы ядра Linux. Режим реального времени. Ленивое вытеснение 5
1.5 Почему нет привычного `printf`? Как всё-таки вывести результат? 6
1.6 Зачем нужны `copy_to_user` / `copy_from_user`, пометка `__user` и передача через `unsigned long` . 6
1.7 Major и minor номера (`ls /dev`) 7
1.8 Разыменование `NULL` в модуле: что будет? Почему kernel panic на любую ошибку? (CrowdStrike) 7
1.9 Файловые системы `/proc`, `/dev`, `/sys`, `/sys/kernel/tracing`: чем отличаются от `/bin`, `/home` 8
2 Вопросы по моему варианту: syscall + `task_struct` 9
2.1 Как проходит системный вызов на x86-64 9
2.2 Как добавить свой системный вызов 9
2.3 Почему модуль не может сам добавить системный вызов 9
2.4 Как заглушка в ядре связана с модулем и что будет при `rmmod` 9
2.5 Что такое `task_struct` 10
2.6 Как найти `task_struct` по PID и почему нужны ссылки и RCU 10
2.7 Почему наружу копируется своя структура, а не `task_struct` 10
2.8 Коды ошибок моего вызова 10
2.9 Почему syscall, а не ioctl или procfs 11
3 Что спросят про мой код (uВопросы) 12
4 Команды для демонстрации 13
5 Материалы 14

1 Официальные вопросы на понимание

1.1 Драйверы устройств: отличие от обычных программ, ограничения и возможности, безопасность, сборка

> КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ
> Драйвер — код в ядре, который реализует стандартный интерфейс ядра (`file_operations`, блочный, сетевой) для конкретного устройства. Он работает в режиме ядра (ring 0) в общем адресном пространстве с ядром: может всё, но любая ошибка роняет всю систему. Собирается системой kbuild против исходников именно той версии ядра, в которую будет загружен.

Чем отличается от обычной программы.

- Нет `main` и своего процесса. Драйвер регистрирует обработчики при загрузке и дальше вызывается ядром: в контексте процесса, который сделал системный вызов, в контексте прерывания или в потоке ядра.
- Нет libc: ни `printf`, ни `malloc`, ни `fopen`. Есть только функции, которые ядро экспортирует (`EXPORT_SYMBOL`): `kmalloc`, `printk`, `copy_to_user`…
- Маленький стек фиксированного размера: 16 КиБ на x86-64. Большие локальные массивы и глубокая рекурсия запрещены.
- Нет защиты памяти: запись мимо буфера портит чужие данные ядра.
- Всё параллельно: SMP, вытеснение, прерывания. Общие данные защищают спинлоками, мьютексами, RCU, атомиками.
- Нет автоматической уборки: всё, что модуль выделил и зарегистрировал, он обязан освободить сам при выгрузке. Утечка живёт до перезагрузки.
- По умолчанию нельзя пользоваться FPU/SIMD (см. 1.3).

Возможности. Доступ к железу (порты ввода-вывода, MMIO, DMA), ко всей памяти и ко всем структурам ядра, перехват событий системы.

Безопасность.

- Загрузить модуль может только процесс с `CAP_SYS_MODULE` (на практике root).
- Подпись модулей (`CONFIG_MODULE_SIG`): при Secure Boot и lockdown неподписанный модуль не загрузится.
- Загрузка внешнего модуля помечает ядро флагами taint: `O` — внешний (out-of-tree), `E` — неподписанный, `P` — несвободная лицензия. По ним разработчики ядра видят, что в багрепорте замешан чужой код.
- Ошибка в драйвере — уязвимость всей системы. Поэтому всё, что приходит от пользователя (числа, размеры, указатели), проверяется: указатели только через `copy_from_user` / `copy_to_user`.

Сборка. Внешний модуль собирается kbuild'ом против build-дерева установленного ядра: `Makefile` с `obj-m += src/vtkm.o` и вызов `make -C /lib/modules/$(uname -r)/build M=$PWD modules` (так сделано в заготовке лабы). Результат — `vtkm.ko`, перемещаемый ELF-файл с секцией `.modinfo`. Модуль подходит только к ядру той же версии и конфигурации: строка `vermagic` должна совпасть, иначе `insmod` ответит `Invalid module format`. Модули внутри дерева ядра включаются в `Kconfig` значениями `y` (вшить в ядро) / `m` (модулем) / `n`.

1.2 Модули ядра Linux: создание и загрузка, отличие от программ в user space

> КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ
> Модуль — кусок ядра в виде `.ko`, который подгружается в работающее ядро без перезагрузки. `insmod` передаёт файл ядру системным вызовом `finit_module`; ядро проверяет его, связывает с экспортированными символами и вызывает функцию из `module_init`. `rmmod` вызывает функцию из `module_exit`, если модулем сейчас никто не пользуется.

Создание. Минимум — две функции и макросы, как в заготовке `lab/vtkm/src/vtkm.c`: `module_init(vtkm_init)`, `module_exit(vtkm_exit)`, `MODULE_LICENSE("GPL")`. Лицензия не формальность: символы, экспортированные через `EXPORT_SYMBOL_GPL`, доступны только GPL-модулям, а модуль с несвободной лицензией ставит taint `P`.

Загрузка по шагам.

1. `insmod vtkm.ko` → системный вызов `finit_module(fd, …)`.
2. Ядро проверяет подпись (если требуется) и `vermagic`, выделяет память, выполняет релокации.
3. Неопределённые символы модуля связываются с таблицей экспортированных символов ядра и других модулей.
4. Вызывается init-функция. Если она вернула ошибку, модуль выгружается, `insmod` сообщает ошибку.
5. Модуль виден в `lsmod` (`/proc/modules`) и в `/sys/module/vtkm/`.

`modprobe` делает то же по имени, сам подгружая зависимости из `modules.dep` (его строит `depmod`). Выгрузка: `rmmod` → `delete_module`; если счётчик ссылок модуля не ноль, будет `EBUSY`.

Отличия от программы в user space. Init-функция отрабатывает и возвращает управление, а код модуля остаётся в ядре до `rmmod`. Нет `main`, libc, своего процесса, изоляции памяти; ошибка — oops или panic, а не «упала одна программа». Вывод — через `printk` в журнал ядра, а не в терминал. Параметры передаются через `module_param` и видны в `/sys/module/<имя>/parameters/`.

> **КАК ПОКАЗАТЬ В VM**
>
> ```
> make && sudo insmod vtkm.ko && sudo dmesg | tail -3
> lsmod | grep vtkm;  modinfo vtkm.ko;  cat /proc/sys/kernel/tainted
> sudo rmmod vtkm && sudo dmesg | tail -1
> ```

1.3 Архитектура ОС: сложности разработки в ядре, нюансы работы с FPU

> **КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ**
>
> Linux — монолитное ядро с загружаемыми модулями: всё ядро и все драйверы в одном адресном пространстве. Это быстро (вызов драйвера — обычный вызов функции), но нет изоляции. Отсюда сложности: отладка, параллелизм, контексты, где нельзя спать, маленький стек, нестабильный внутренний API. FPU в ядре по умолчанию запрещён: его регистры принадлежат пользовательскому процессу.

Сложности разработки.

- **Отладка.** Ошибка роняет саму систему, на которой отлаживаешь. Инструменты: `printk` и dynamic debug, ftrace, kgdb, QEMU + gdb, kdump + crash. Поэтому разработка — в виртуалке.
- **Контексты.** В контексте процесса (системный вызов) спать можно. В контексте прерывания и в атомарных участках (держим спинлок, `rcu_read_lock`, `preempt_disable`) — нельзя: нельзя `kmalloc(GFP_KERNEL)`, мьютекс, `copy_to_user` (может случиться page fault, и тогда придётся ждать).
- **Параллелизм.** Один и тот же код одновременно выполняется на нескольких CPU и может быть вытеснен.
- **Нестабильный внутренний API.** Функции ядра меняются между версиями: например, в 6.4 у `class_create` убрали первый аргумент. Внешний модуль приходится поддерживать под каждую версию.
- **Ресурсы.** Память ядра не уходит в swap, утечки копятся до перезагрузки.

FPU и SIMD. При входе в ядро состояние FPU/SSE/AVX пользовательского процесса не сохраняется: это дорого, а ядру эти регистры обычно не нужны. Если код ядра тронет регистр `xmm0`, он испортит вычисления процесса, который нас вызвал. Поэтому ядро собирается с флагами, запрещающими компилятору FP и SIMD (на x86 — `-mgeneral-regs-only` / `-mno-sse` и подобные). Когда SIMD всё-таки нужен (шифрование AES-NI, RAID6, графика amdgpu), код оборачивают в `kernel_fpu_begin()` /

`kernel_fpu_end()`: ядро сохраняет пользовательское состояние FPU и запрещает вытеснение, поэтому внутри нельзя спать. Такой код выносят в отдельный файл с особыми флагами компиляции (docs.kernel.org: Floating-point API).

1.4 Режимы работы ядра Linux. Режим реального времени. Ленивое вытеснение

> **КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ**
> Процессор работает в режиме пользователя (ring 3) или ядра (ring 0). Внутри ядра код выполняется в контексте процесса или прерывания. «Режим работы ядра» в смысле настроек — модель вытеснения: можно ли отобрать CPU у кода, который сейчас выполняется в ядре. Модели: none, voluntary, full, lazy и real-time (PREEMPT_RT). Компромисс — пропускная способность против задержки отклика.

Режимы процессора и контексты. В ядро попадают по инструкции `syscall`, по прерыванию или исключению (page fault). Контекст процесса — системный вызов и потоки ядра, здесь можно спать. Контекст прерывания (hardirq, softirq) — спать нельзя, `current` здесь не имеет отношения к происходящему.

Модели вытеснения (`CONFIG_PREEMPT_*`):

none	Код ядра не вытесняется принудительно. Переключение только при возврате в user space или когда код сам блокируется. Максимум пропускной способности, большие задержки. Серверы.
voluntary	То же плюс точки добровольного вытеснения (`cond_resched()`, `might_sleep()`) в длинных циклах. Десктопы.
full	Ядро можно вытеснить в любом месте, кроме участков с запрещённым вытеснением: держим спинлок, `preempt_disable()`, выключены прерывания (счётчик `preempt_count`). Низкая задержка.
lazy (с 6.13)	Для обычных задач (`SCHED_NORMAL`) вытеснение откладывается до возврата в user space или до следующего тика таймера: почти как none по пропускной способности. Если задача сидит в ядре дольше тика, вытеснение становится полным. Задачи реального времени вытесняют сразу, как в full. Цель — избавиться от `cond_resched()`, расставленных по всему ядру, и от моделей none/voluntary.
PREEMPT_RT (в основной ветке с 6.12)	Реальное время: спинлоки превращаются в спящие `rt_mutex` с наследованием приоритета, обработчики прерываний выполняются в потоках ядра. Почти всё ядро вытесняемо, задержка ограничена и предсказуема (десятки микросекунд). Нужно для станков, звука, роботов.

Реальное время — это не «быстро», а «предсказуемо»: гарантированная верхняя граница задержки.

С ядра 7.0 на большинстве архитектур, включая x86, остались только две модели: **lazy** и **full**. Voluntary доступна только там, где нет lazy, none — где вытеснения нет вообще (LWN, окно слияния 7.0).

> **КАК ПОКАЗАТЬ В VM**

```
uname -v        # в строке: PREEMPT_DYNAMIC / PREEMPT_LAZY / PREEMPT_RT
sudo cat /sys/kernel/debug/sched/preempt   # текущая модель в скобках
```

1.5 Почему нет привычного `printf`? Как всё-таки вывести результат?

> КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ
> `printf` — функция libc: форматирует строку и делает `write(1, …)` в stdout процесса. Ядро не слинковано с libc, у него нет своего stdout, а вызвать системный вызов само у себя ему незачем. В ядре есть `printk`: он пишет сообщение в кольцевой буфер журнала ядра, откуда его читают `dmesg` и `journalctl -k`.

- `printk` работает в любом контексте, даже в прерывании: он не спит.
- У каждого сообщения есть уровень от `KERN_EMERG` (0) до `KERN_DEBUG` (7). Обычно пишут макросами `pr_err`, `pr_info`, `pr_debug`; в заготовке лабы макрос `LOG` — это `pr_info` с префиксом `[vtkm]: `.
- Сообщения важнее порога `console_loglevel` дополнительно печатаются на консоль; пороги — в `/proc/sys/kernel/printk`.
- `pr_debug` по умолчанию не выводится, его включают через dynamic debug: `/sys/kernel/debug/dynamic_debug/control`.
- Указатель по `%p` печатается хешированным, чтобы не раскрывать адреса ядра (защита KASLR).
- Сообщение должно заканчиваться `\n`; в горячих местах — `printk_ratelimited`.

Вывести результат **пользователю** в терминал из ядра нельзя. В моей лабе модуль копирует данные в буфер программы через `copy_to_user`, и печатает их уже программа в user space обычным `printf`.

1.6 Зачем нужны `copy_to_user` / `copy_from_user`, пометка `__user` и передача через `unsigned long`

> КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ
> Указатель от пользователя — адрес в его адресном пространстве, и доверять ему нельзя: он может быть NULL, указывать на невыделенную память или **в память ядра** (тогда ядро по просьбе пользователя запишет в себя — повышение привилегий). `copy_to_user` проверяет, что диапазон лежит в пользовательской части, и копирует так, что page fault не роняет ядро, а превращается в код ошибки.

Что делает `copy_to_user(to, from, n)`.

- `access_ok`: диапазон целиком в пользовательской части адресного пространства.
- Временно разрешает доступ к пользовательским страницам: процессоры с SMAP (x86) и PAN (arm64) аппаратно запрещают ядру трогать память пользователя без явного разрешения (`stac` / `clac`). Прямое разыменование пользовательского указателя на таких CPU — oops.
- Копирует с записью в таблицу исключений: если страница не отображена, обработчик page fault не паникует, а копирование прерывается.
- Возвращает число **не скопированных** байтов: 0 — успех, иначе системный вызов возвращает `-EFAULT`.
- Может спать (подкачка страницы), поэтому нельзя вызывать под спинлоком или внутри `rcu_read_lock()`.

> УТЕЧКА ЧЕРЕЗ PADDING
> Структуру, которую копируем пользователю, сначала обнуляют (`memset` или `= {}`), иначе в байтах выравнивания между полями уйдут куски старого содержимого стека ядра.

`__user` — пометка для статического анализатора sparse (`make C=1`). Компилятор её игнорирует, а sparse ругается, если `__user`-указатель разыменован напрямую или смешан с указателем ядра.

`unsigned long`. В `ioctl(fd, cmd, unsigned long arg)` третий аргумент универсальный: это может быть число или указатель, драйвер сам приводит его к `(void __user *)arg`. У системных вызовов так же: на x86-64 все шесть аргументов передаются в 64-битных регистрах (`rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`), указатель туда помещается. Макросы `SYSCALL_DEFINEn` принимают регистры как `long` и приводят к объявленным типам; обёртки появились после CVE-2009-0029 — ошибки знакового расширения 32-

битных аргументов на 64-битных платформах. В моём варианте: `SYSCALL_DEFINE3(vtkm, pid_t, pid, struct vtkm_info __user *, buf, size_t, size)`.

1.7 Major и minor номера (`ls /dev`)

> **КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ**
> Файлы устройств в `/dev` — специальные файлы символьных (`c`) и блочных (`b`) устройств. Вместо размера `ls -l` показывает два числа: major — какой драйвер обслуживает файл, minor — какое устройство из обслуживаемых этим драйвером.

```
$ ls -l /dev/null /dev/sda /dev/sda1
crw-rw-rw- 1 root root 1, 3 ... /dev/null
brw-rw---- 1 root disk 8, 0 ... /dev/sda
brw-rw---- 1 root disk 8, 1 ... /dev/sda1
```

- Внутри ядра это `dev_t`: 12 бит major, 20 бит minor; макросы `MAJOR()`, `MINOR()`, `MKDEV()`.
- Драйвер получает номера через `alloc_chrdev_region` (major выделяется динамически) или `register_chrdev_region` и регистрирует устройство `cdev_add`. Файл в `/dev` создаёт udev по событию `device_create` или вручную `mknod`.
- Занятые major — в `/proc/devices`, официальный реестр — admin-guide/devices.
- При `open()` ядро по major/minor находит драйвер и вызывает его `file_operations`.

В моём варианте файл устройства не нужен: связь идёт через системный вызов. Он понадобился бы в вариантах `ioctl`.

1.8 Разыменование `NULL` в модуле: что будет? Почему kernel panic на любую ошибку? (CrowdStrike)

> **КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ**
> Нулевая страница не отображена, поэтому будет page fault в режиме ядра без обработчика — **oops**: «BUG: kernel NULL pointer dereference», регистры, стек вызовов, ядро помечается taint `D`. В контексте процесса ядро убивает текущий процесс и продолжает работу в неизвестном состоянии. В прерывании, в потоке idle/init или при `panic_on_oops=1` — **kernel panic**, система останавливается.

Почему ядро не может просто «закрыть программу». Пользовательский процесс изолирован: если он упал, его память и ресурсы можно выбросить, остальным это не мешает. У ядра изоляции нет: сбойный код мог испортить общие структуры, держать блокировку, которая теперь не освободится, или оставить неверный счётчик ссылок. Какие данные повреждены, неизвестно. Продолжать опасно: можно, например, записать мусор на диск. Поэтому надёжнее остановиться (panic) и перезагрузиться.

Почему NULL не отображён: параметр `vm.mmap_min_addr` запрещает пользователю отображать страницы у нуля. Раньше атаки так и работали: пользователь клал свой код по адресу 0, а ядро разыменовывало NULL.

CrowdStrike, 19 июля 2024. Антивирус Falcon работает в Windows как драйвер ядра (`csagent.sys`). Пришло обновление контента «Channel File 291»: шаблон ожидал 21 поле входных данных, а передано было 20, и драйвер прочитал память за границей массива. Чтение в ядре по неверному адресу дало BSOD — аналог kernel panic в Windows. Драйвер загружается при старте системы, поэтому машины падали снова при каждой загрузке. Пострадало около 8,5 млн компьютеров; лечили вручную через безопасный режим, удаляя файл (разбор CrowdStrike). Мораль: код ядра обязан проверять любые входные данные; поэтому такую логику всё чаще переносят в eBPF, где программу перед загрузкой проверяет верификатор.

СВЯЗЬ С МОИМ КОДОМ

У потоков ядра `task->mm == NULL`. Если моя программа запросит PID потока ядра (например, `kthreadd`, PID 2), а модуль без проверки прочитает `task->mm->total_vm`, это и будет разыменование NULL.

1.9 Файловые системы `/proc`, `/dev`, `/sys`, `/sys/kernel/tracing`: чем отличаются от `/bin`, `/home`

КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ

`/bin` и `/home` — каталоги на настоящей файловой системе (ext4, btrfs): данные лежат блоками на диске и переживают перезагрузку. `/proc`, `/sys`, debugfs, tracefs, `/dev` — виртуальные файловые системы: на диске их нет, содержимое генерирует ядро в момент чтения, а запись — это команда ядру. Это интерфейс к ядру в виде файлов: «всё есть файл».

Где	ФС	Что там
`/proc`	procfs	Процессы (`/proc/<pid>/status`, `maps`, `fd/`), сведения о системе (`meminfo`, `cpuinfo`), настройки ядра `/proc/sys` (sysctl).
`/sys`	sysfs	Модель устройств ядра: устройства, драйверы, шины, модули. Правило: одно значение в файле.
`/dev`	devtmpfs	Файлы устройств с major/minor, через них программы обращаются к драйверам.
`/sys/kernel/debug`	debugfs	Для отладки, без правил формата, только root.
`/sys/kernel/tracing`	tracefs	Интерфейс ftrace: трассировка функций и событий ядра.

У виртуальных файлов `ls -l` обычно показывает размер 0: содержимого заранее нет. Список смонтированных: `mount | grep -E 'proc|sysfs|devtmpfs|debugfs|tracefs'`.

В моём варианте вместо этих файловых систем используется системный вызов. Зато `/proc/<pid>/status` нужен для проверки: вывод моей программы должен совпасть с ним.

2 Вопросы по моему варианту: syscall + `task_struct`

2.1 Как проходит системный вызов на x86-64

> **КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ**
> Программа кладёт номер вызова в `rax`, аргументы в `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` и выполняет инструкцию `syscall`. Процессор переходит в ring 0 на точку входа `entry_SYSCALL_64`, ядро сохраняет регистры в `pt_regs` на стеке ядра и вызывает обработчик по номеру. Результат возвращается в `rax`; ошибка — отрицательный код `-errno`.

1. Своего вызова нет в libc, поэтому зову `syscall(__NR_vtkm, pid, &buf, sizeof buf)` (`man 2 syscall`).
2. `syscall` → `entry_SYSCALL_64` (адрес задан в MSR `LSTAR`) → `do_syscall_64`.
3. С 2024 года на x86-64 обработчик выбирается через `switch` в `x64_sys_call(regs, nr)`, а не по массиву `sys_call_table` (защита от Spectre: косвенный вызов заменён условными переходами).
4. `__x64_sys_vtkm(regs)` достаёт аргументы из `pt_regs`, приводит типы и вызывает тело, которое я написал внутри `SYSCALL_DEFINE3`.
5. Возврат через `sysret`. Если ядро вернуло, например, `-ESRCH`, libc кладёт `ESRCH` в `errno` и возвращает `-1`.

2.2 Как добавить свой системный вызов

- Строка в `arch/x86/entry/syscalls/syscall_64.tbl`: `<номер> common vtkm sys_vtkm`. Номер — следующий свободный в таблице моей версии ядра.
- Тело — `SYSCALL_DEFINE3(vtkm, …)` в файле ядра (например, `kernel/vtkm.c` + строка `obj-y` в `kernel/Makefile`).
- Прототип `asmlinkage long sys_vtkm(…)` в `include/linux/syscalls.h`.
- Пересобрать и установить ядро, загрузиться в него. Модуль собирать уже против нового ядра.

Порядок описан в Adding a New System Call.

2.3 Почему модуль не может сам добавить системный вызов

- `sys_call_table` не экспортирована для модулей.
- С 5.7 не экспортирована и `kallsyms_lookup_name`, через которую таблицу находили.
- С 5.3 на x86 бит защиты записи в `CR0` закреплён (CR pinning): снять его обычной функцией нельзя.
- С 2024 года таблица на x86-64 вообще не используется для вызова обработчиков (`x64_sys_call`): подмена её слота ничего не даёт.

Поэтому выбран способ «заглушка в пересобранном ядре + обработчик в модуле». Перехват через kprobe или ftrace возможен, но это хак, зависящий от деталей реализации.

2.4 Как заглушка в ядре связана с модулем и что будет при `rmmod`

> **КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ**
> Ядро экспортирует функцию регистрации (`EXPORT_SYMBOL_GPL`). Модуль при загрузке передаёт ей свой обработчик и снимает его при выгрузке. Пока обработчика нет, заглушка возвращает `-ENOSYS`. Главная опасность — выгрузка модуля во время вызова: код обработчика исчезнет из памяти.

Типовое решение: хранить указатель на структуру `{ обработчик, owner = THIS_MODULE }`. В заглушке: `rcu_read_lock()`, прочитать указатель, `try_module_get(owner)`, `rcu_read_unlock()`, вызвать обработчик, `module_put(owner)`. Пока счётчик ссылок модуля больше нуля, `rmmod` откажет с `EBUSY`. При выгрузке модуль обнуляет указатель и вызывает `synchronize_rcu()`: после этого никто не находится между чтением указателя и `try_module_get`. Под `rcu_read_lock()` сам обработчик вызывать нельзя: внутри него `copy_to_user`, который может спать.

2.5 Что такое `task_struct`

> **КОРОТКО — ЭТО СКАЗАТЬ ПЕРВЫМ**
> Дескриптор задачи: всё, что ядро знает о потоке. У каждого **потока** свой `task_struct`; `pid` в нём — идентификатор потока, `tgid` — идентификатор процесса (то, что пользователь видит как PID). Определена в `include/linux/sched.h`, весит несколько килобайт, выделяется из slab-кеша. Текущая задача — макрос `current`.

Поля, которые я вывожу и должен уметь объяснить:

`comm`	Имя исполняемого файла, до 16 байт (`TASK_COMM_LEN`).
`pid`, `tgid`	ID потока и ID процесса (группы потоков).
`real_parent`	Родитель; его `tgid` — PPID.
`__state`	Состояние: выполняется/готов (`TASK_RUNNING`), спит с прерыванием (`TASK_INTERRUPTIBLE`), без (`TASK_UNINTERRUPTIBLE`) и т.д.
`prio`, `static_prio`, `normal_prio`	Приоритеты планировщика: `static_prio` задаётся nice (120 + nice), `prio` — действующий с учётом наследования.
`policy`	Политика: `SCHED_NORMAL`, `SCHED_FIFO`, `SCHED_RR`, `SCHED_BATCH`, `SCHED_IDLE`, `SCHED_DEADLINE`.
`utime`, `stime`	Время CPU в режиме пользователя и ядра, в наносекундах.
`nvcsw`, `nivcsw`	Добровольные (заблокировался сам) и принудительные (вытеснили) переключения контекста.
`mm`	Адресное пространство (`mm_struct`): `total_vm`, границы кода, кучи, стека. **У потоков ядра `NULL`.**

2.6 Как найти `task_struct` по PID и почему нужны ссылки и RCU

- PID — число в пространстве имён PID вызывающего процесса. `find_get_pid(nr)` находит `struct pid` в таблице PID (IDR пространства имён) и увеличивает её счётчик ссылок.
- `get_pid_task(pid, PIDTYPE_PID)` возвращает `task_struct` с увеличенным счётчиком ссылок.
- После работы — `put_task_struct(task)` и `put_pid(pid)`. Без этого утечка; лишний `put` — use-after-free.
- Зачем ссылки: процесс может завершиться прямо во время моего вызова. `task_struct` освобождается через RCU после завершения; ссылка не даёт освободить структуру, пока я её читаю.
- Перебор всех задач (`for_each_process`) тоже работает, но это O(n) и требует `rcu_read_lock`.
- Поля могут меняться, пока я их читаю: снимок не атомарный. Для информационного вывода это допустимо.

2.7 Почему наружу копируется своя структура, а не `task_struct`

- Раскладка `task_struct` зависит от версии и конфигурации ядра: программа не сможет её разобрать.
- Внутри указатели ядра: их выдача раскрывает адреса и помогает обойти KASLR.
- Она огромная, а нужна малая часть полей.

Поэтому общий заголовок с моей структурой ответа подключают и модуль, и программа. Типы фиксированного размера (`__u32`, `__u64`), чтобы раскладка совпадала; перед заполнением — обнуление (см. 1.6).

2.8 Коды ошибок моего вызова

`-ENOSYS`	Модуль не загружен, обработчик не зарегистрирован.
`-ESRCH`	Процесса с таким PID нет.
`-EFAULT`	`copy_to_user` не смог записать: плохой указатель на буфер.

`-EINVAL`	Неверный размер буфера или аргумент (например, PID ⩽ 0).

Программа получает `-1` и `errno`, сообщение печатает через `perror` / `strerror`.

2.9 Почему syscall, а не ioctl или procfs

Интерфейс	**Плюсы**	**Минусы**
syscall	Быстрый, типизированные аргументы, без файла устройства.	Нужна пересборка ядра и номер вызова; номера — общий ресурс, в реальном ядре новый вызов — серьёзное решение.
ioctl	Без пересборки ядра, через файл устройства.	Нужен символьный драйвер, `arg` не типизирован.
procfs / debugfs	Просто: `echo` и `cat`, текстовый формат.	Текст надо разбирать, два шага (записать запрос, прочитать ответ), состояние между ними.

3 Что спросят про мой код (uВопросы)

Официального списка пока нет. Эти вопросы проверяют, что код написан самостоятельно. Ответы зависят от моего кода — готовь их по нему.

- Покажи, где в коде ищется процесс по PID. Какие функции и почему именно они?
- Где освобождаются ссылки? Что будет, если убрать `put_task_struct`?
- Что вернёт программа для PID 2 (`kthreadd`)? Для несуществующего PID? Для PID потока, а не процесса?
- Что будет, если вызвать системный вызов, когда модуль не загружен? Если выгрузить модуль во время вызова?
- Какой номер у моего системного вызова и где он задан? Покажи патч ядра.
- Почему структура ответа обнуляется перед заполнением?
- Откуда взялось каждое поле в выводе и как проверить его через `/proc/<pid>/status` или `ps`?
- Что в `Makefile` значит `obj-m` и `-C $(KDIR) M=$(PWD)`? Почему модуль собран против нового ядра?
- Что напишет `dmesg` при загрузке модуля и почему там слова про taint?
- Какие промпты к ИИ использовались (прикладываются к отчёту).

4 Команды для демонстрации

```
# модуль
make && sudo insmod vtkm.ko
sudo dmesg -w                       # журнал ядра в реальном времени
lsmod | grep vtkm; modinfo vtkm.ko; ls /sys/module/vtkm
cat /proc/sys/kernel/tainted        # флаги taint числом
sudo rmmod vtkm

# мой вызов и сверка
./vtkm-cli $$                       # PID текущего shell
grep -E 'Name|State|Tgid|PPid|Threads|ctxt' /proc/$$/status
ps -o pid,ppid,pri,ni,cls,stat,comm -p $$
./vtkm-cli 2; ./vtkm-cli 999999     # поток ядра и несуществующий PID

# ядро
uname -r; uname -v
sudo cat /sys/kernel/debug/sched/preempt
sudo grep vtkm /proc/kallsyms   # символы заглушки и модуля
cat /proc/sys/kernel/printk
ls -l /dev | head; cat /proc/devices | head
mount | grep -E 'proc|sysfs|devtmpfs|debugfs|tracefs'
```

5 Материалы

Задание и вопросы
- Условие: github.com/secs-dev/os-course/lab/vtkm
- Вопросы: вкладка «LNX-4: VTKM» в документе «Задания ОС 2026»
- Порядок сдачи: doc/process.md

Модули ядра
- The Linux Kernel Module Programming Guide и перевод на русский
- Linux Kernel Labs: Kernel modules, Kernel API, Character device drivers
- Building External Modules (kbuild)
- Linux Device Drivers, 3rd ed. — старая, но базовые идеи те же

Системные вызовы
- Adding a New System Call
- Linux Kernel Labs: лекция System Calls
- LWN «Anatomy of a system call»: часть 1, часть 2
- `man`: syscall(2), syscalls(2)
- Исходники: syscall_64.tbl, syscall_64.c

`task_struct` и PID
- include/linux/sched.h, kernel/pid.c
- What is RCU?

Сборка ядра
- How to quickly build a trimmed Linux kernel
- Ubuntu: Build a kernel
- virtme-ng — запуск собранного ядра в QEMU

К вопросам защиты
- printk basics, Floating-point API, Linux allocated devices
- procfs, sysfs, debugfs, ftrace
- Tainted kernels, Bug hunting (разбор oops)
- LWN: The long road to lazy preemption, окно слияния 7.0
- CrowdStrike: Root Cause Analysis, Channel File 291

Видео
- АКОС-20. Модули ядра и FUSE (Лекторий ФПМИ)
- Let’s code a Linux Driver — 1: Hello World Linux Kernel Module
- How to Create a Custom Linux System Call